

EDS Theory :

Assignment No.2

Name : Pranil Pramod Sangale

PRN : 202501080077

Subject : EDS

Formulate 20 problem statements for a given dataset using Numpy and Pandas and Apply Numpy and pandas methods to find the solution for the formulated problem statements.

Dataset : SMS Spam Collection

Step 1: Import Libraries and load Dataset


Untitled13.ipynb
☆
🔗

File Edit View Insert Runtime Tools Help

+ Code
+ Text
▶ Run all

☰

🔍

⏮

🔍

🔑

📁

📊

[1] ✓ 14s

▶

```
from google.colab import files
uploaded = files.upload()
```

⋮

Choose Files

spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done

Saving spam (1).csv to spam (1).csv

[12] ✓ 0s

▶

```
import pandas as pd
import numpy as np

# Load the correct file (your uploaded file)
df = pd.read_csv('spam (1).csv', encoding='latin-1')

df.head(10)
```

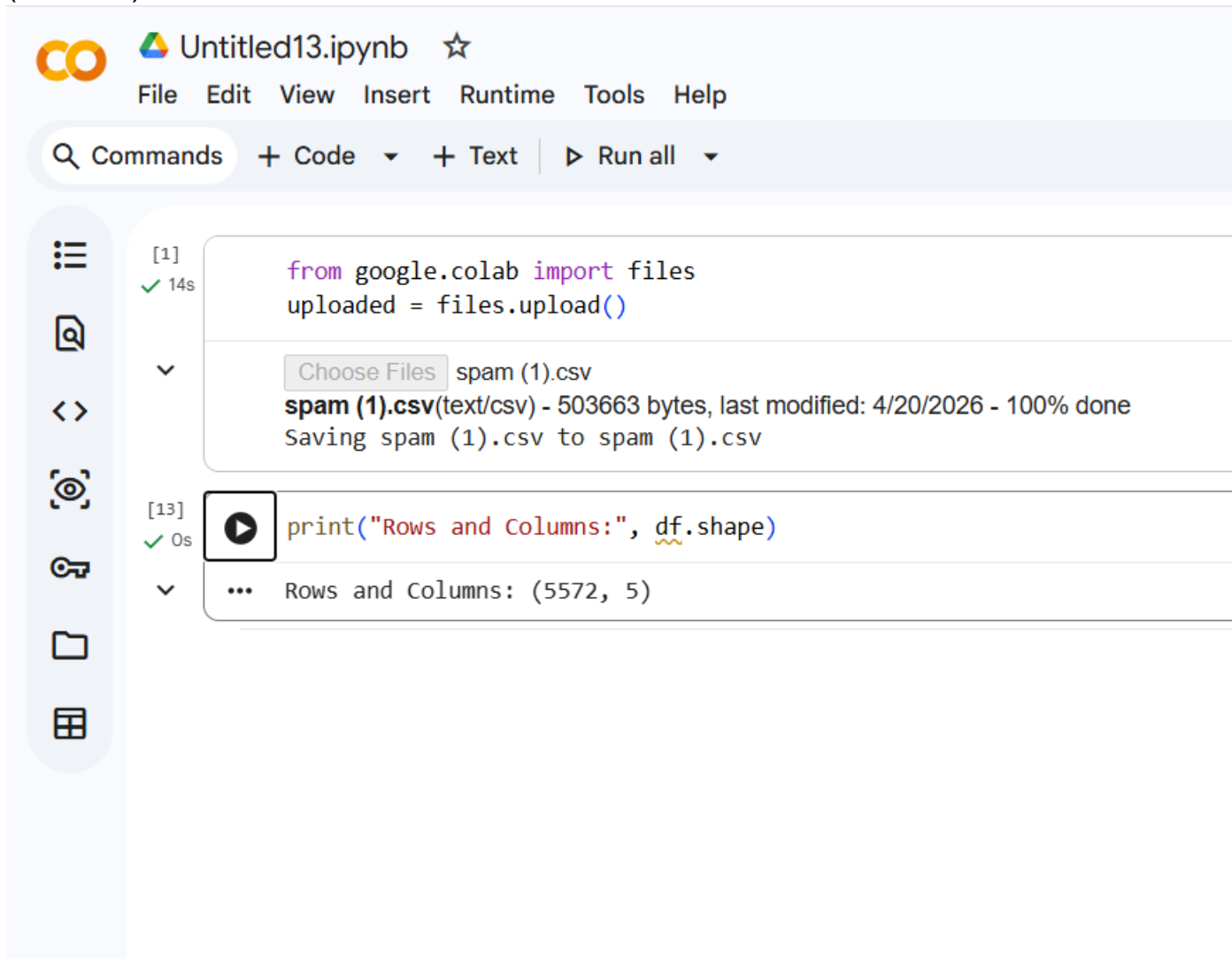
⋮

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN
5	spam	FreeMsg Hey there darling it's been 3 week's n...	NaN	NaN	NaN
6	ham	Even my brother is not like to speak with me. ...	NaN	NaN	NaN
7	ham	As per your request 'Melle Melle (Oru Minnamin...	NaN	NaN	NaN
8	spam	WINNER!! As a valued network customer you have...	NaN	NaN	NaN
9	spam	Had your mobile 11 months or more? U R entitle...	NaN	NaN	NaN

Step 2 : 20+ Problem Statements and Solutions

1. Shape of dataset

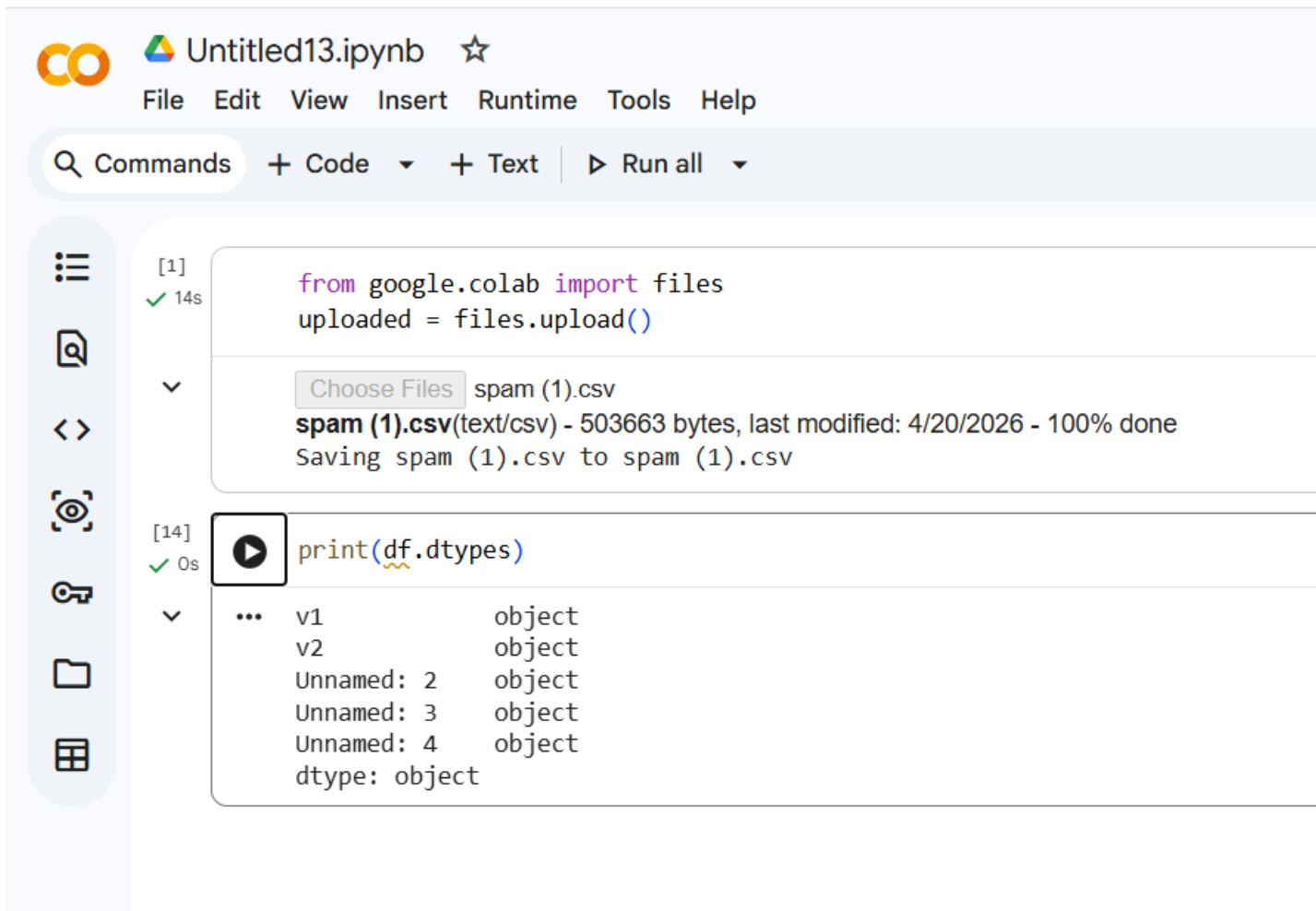
Problem Statement: Find the total number of messages (rows) and features (columns) in the dataset.



The screenshot shows a Google Colab notebook interface. At the top, the title bar reads 'Untitled13.ipynb' with a star icon. Below it is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar contains a search icon, 'Commands', '+ Code', '+ Text', and a 'Run all' button. On the left is a vertical sidebar with icons for a menu, search, code editor, viewer, keychain, folder, and table. The main area displays two code cells. The first cell, labeled '[1]' with a green checkmark and '14s', contains the code `from google.colab import files` and `uploaded = files.upload()`. Below the code is a file upload dialog showing 'spam (1).csv' with a 'Choose Files' button. A status message indicates 'spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done' and 'Saving spam (1).csv to spam (1).csv'. The second cell, labeled '[13]' with a green checkmark and '0s', contains the code `print("Rows and Columns:", df.shape)`. Below the code is a play button icon and the output '... Rows and Columns: (5572, 5)'.

2. DATA TYPES

PROBLEM STATEMENT: Display the column names and their data types.



The screenshot shows a Google Colab notebook titled "Untitled13.ipynb". The interface includes a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu is a "Commands" search bar and buttons for "+ Code", "+ Text", and "Run all". On the left sidebar, there are icons for file management and execution. The notebook contains two code cells. The first cell, labeled "[1]" and "14s", contains the code `from google.colab import files` and `uploaded = files.upload()`. Below the code, a file named "spam (1).csv" is shown as uploaded, with a size of 503663 bytes and a status of "100% done". The second cell, labeled "[14]" and "0s", contains the code `print(df.dtypes)`. The output of this cell shows the data types for columns v1, v2, Unnamed: 2, Unnamed: 3, Unnamed: 4, and dtype, all of which are "object".

```
[1] ✓ 14s
from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[14] ✓ 0s
print(df.dtypes)

... v1          object
    v2          object
    Unnamed: 2   object
    Unnamed: 3   object
    Unnamed: 4   object
    dtype: object
```

3 MISSING VALUES

PROBLEM STATEMENT: Check how many missing values are present in each column.

 **Untitled13.ipynb** ☆ ↺ Saving...
File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾









[1]
✓ 14s

```
from google.colab import files  
uploaded = files.upload()
```

▼

Choose Files spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[15]
✓ 0s



```
print(df.isnull().sum())
```

▼
... v1 0
v2 0
Unnamed: 2 5522
Unnamed: 3 5560
Unnamed: 4 5566
dtype: int64

4. GROUP STATISTICS

PROBLEM STATEMENT: Group the dataset by label and calculate count, mean message length.

 Untitled13.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[17]
✓ 0s

import pandas as pd

df = pd.read_csv('spam (1).csv', encoding='latin-1')

Fix columns
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

Add length column
df['length'] = df['message'].apply(len)

Groupby
print(df.groupby('label')['length'].mean())

... label
ham 71.023627
spam 138.866131
Name: length, dtype: float64

5. LONGEST AND SHORTEST MESSAGE

PROBLEM STATEMENT: Determine the longest and shortest message in the dataset.


Untitled13.ipynb


File Edit View Insert Runtime Tools Help

+ Code
+ Text
▶ Run all

☰

🔍

⏪ ⏩

🔍

[1]

✓ 14s

▼

Choose Files

spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done

Saving spam (1).csv to spam (1).csv

[18]

✓ 0s

▶

```
print("Longest:\n", df.loc[df['length'].idxmax()])
print("Shortest:\n", df.loc[df['length'].idxmin()])
```

Run cell (Ctrl+Enter)

cell executed since last change

executed by Pranil Sangale

10:23 PM (0 minutes ago)

executed in 0.172s

ham

For me the love should start with attraction.i...

910

Name: 1084, dtype: object

Shortest:

label ham

message Ok

length 2

Name: 1924, dtype: object

6. AVERAGE MESSAGE LENGTH

PROBLEM STATEMENT: Find the average message length for spam and ham messages separately.

MIT Academy of Engineering

 **Untitled13.ipynb** ☆
File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾









[1]
✓ 14s

▶ `from google.colab import files`
`uploaded = files.upload()`

⋮ spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[19]
✓ 0s

▶ `print(df.groupby('label')['length'].agg(['count', 'mean']))`

⋮

	count	mean
label		
ham	4825	71.023627
spam	747	138.866131

7. Top 10 frequent words in spam

PROBLEM STATEMENT: Find the top 10 most frequent words in spam messages.

CO

Untitled13.ipynb

☆

FileEditViewInsertRuntimeToolsHelp

Q Commands+ Code+ Text▶ Run all

☰

🔍

⏪

🎯

🔑

📁

📊

[1]
✓ 14s

from google.colab import files

uploaded = files.upload()

Choose Files spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done

Saving spam (1).csv to spam (1).csv

[21]
✓ 0s

▶

Step 1: Clean text

import re

df['clean_msg'] = df['message'].apply(lambda x: re.sub(r'[^\a-zA-Z0-9]',

df['clean_msg'] = df['clean_msg'].str.lower()

Step 2: Extract spam words

spam_words = ' '.join(df[df['label']=='spam']['clean_msg']).split()

Step 3: Count words

word_series = pd.Series(spam_words)

print(word_series.value_counts().head(10))

...

to	686
a	376
call	347
you	287
your	263
free	216
the	204
for	203
now	189
or	188

Name: count, dtype: int64

8. Percentage of spam

PROBLEM STATEMENT: Find the percentage of spam messages in the dataset.

 Untitled13.ipynb ☆

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

▼

Choose Files

spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[23]
✓ 0s

▶ spam_percent = (df['label']=='spam').mean()*100
print("Spam %:", spam_percent)

▼

... Spam %: 13.406317300789663

9. MEAN,MEDIAN,STD

PROBLEM STATEMENT: Convert the message length column into a NumPy array and compute mean, median, and standard deviation.

 **Untitled13.ipynb**  

File Edit View Insert Runtime Tools Help

+ Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

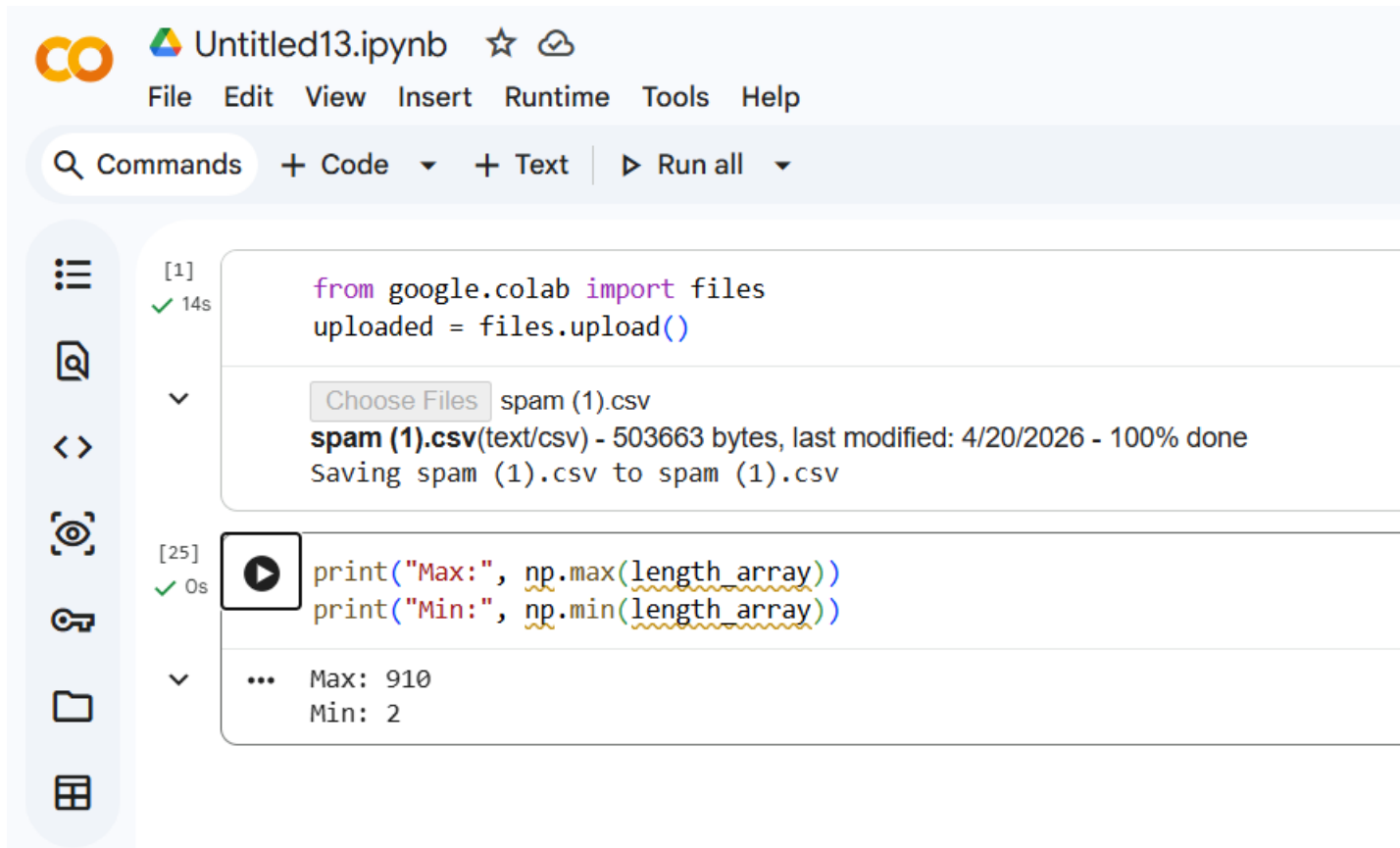
[24]
✓ 0s

 length_array = np.array(df['length'])
print("Mean:", np.mean(length_array))
print("Median:", np.median(length_array))
print("Std:", np.std(length_array))

... Mean: 80.11880832735105
Median: 61.0
Std: 59.6854842151988

10. MAX , MIN

PROBLEM STATEMENT: Find the maximum and minimum message length using NumPy.



CO Untitled13.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

[1] ✓ 14s

```
from google.colab import files
uploaded = files.upload()
```

Choose Files spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[25] ✓ 0s

```
print("Max:", np.max(length_array))
print("Min:", np.min(length_array))
```

... Max: 910
Min: 2

11. NORMALIZE LENGTH

PROBLEM STATEMENT: Normalize the message length using NumPy operations.

CO Untitled13.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all ▼

[1] ✓ 14s

```
from google.colab import files
uploaded = files.upload()
```

... Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[26] ✓ 0s

```
normalized = (length_array - np.min(length_array)) / (np.max(length_array) - np.min(length_array))
print(normalized[:10])
```

... [0.12004405 0.02973568 0.1685022 0.05176211 0.06497797 0.16079295
0.08259912 0.17400881 0.17180617 0.16740088]

12. RESHAPE ARRAY

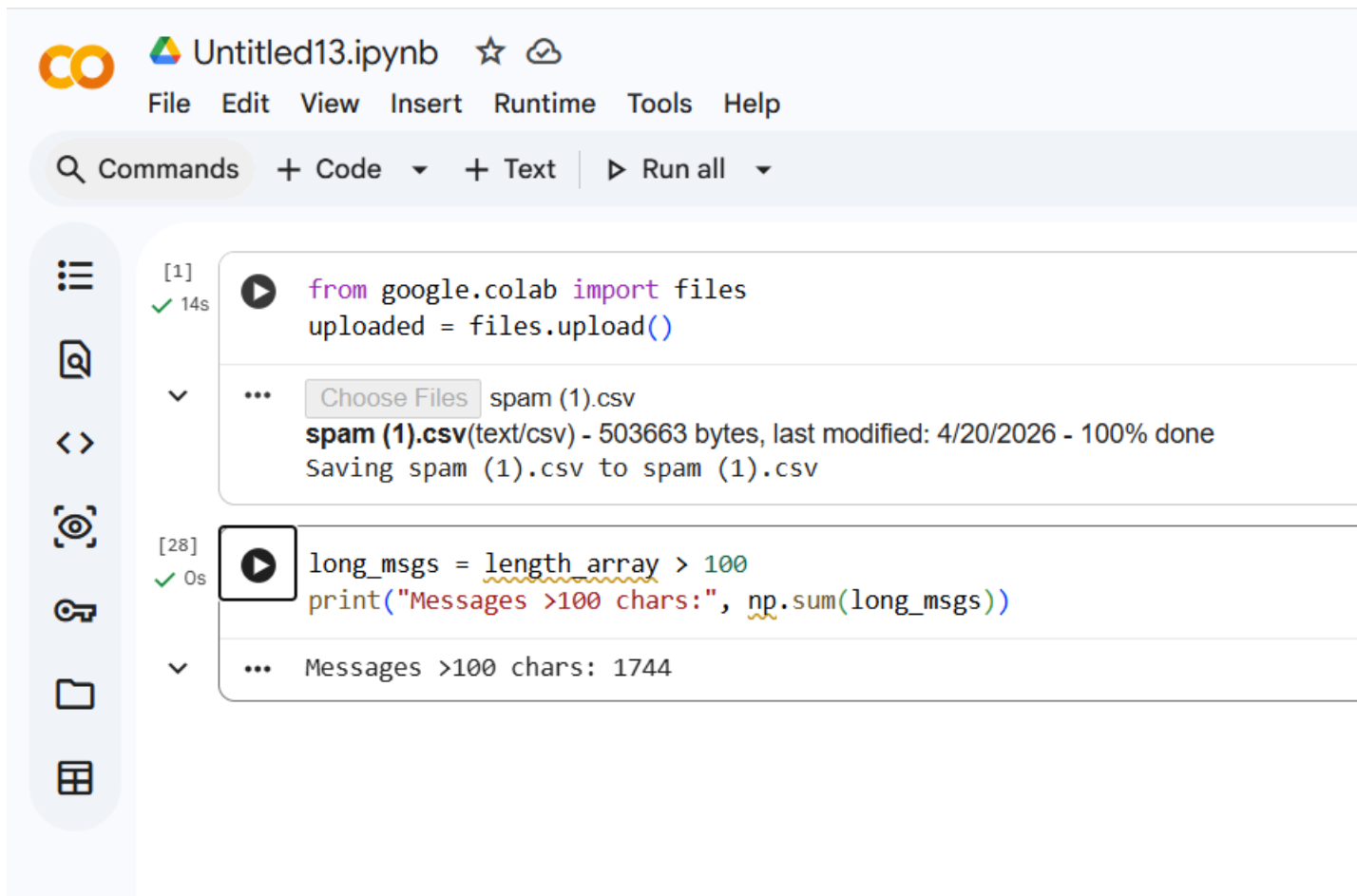
PROBLEM STATEMENT: Create a NumPy array of message lengths and reshape it into a matrix.



The screenshot shows a Google Colab interface. At the top, the notebook is titled "Untitled13.ipynb" and is in the process of saving. Below the title bar is a menu with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar labeled "Commands" is followed by buttons for "+ Code", "+ Text", and a "Run all" button. On the left side, there is a vertical toolbar with icons for a list, a chat bubble, code and text symbols, a camera, a key, a folder, and a table. The main area contains two code cells. The first cell, labeled "[1]" and "14s", contains the code `from google.colab import files` and `uploaded = files.upload()`. Below the code, a file selection dialog shows "spam (1).csv" with a status message: "spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done" and "Saving spam (1).csv to spam (1).csv". The second cell, labeled "[27]" and "0s", contains the code `reshaped = length_array.reshape(-1, 1)` and `print(reshaped[:5])`. Below the code, the output is displayed as a list of five sub-arrays: `[[111]`, `[ 29]`, `[155]`, `[ 49]`, and `[ 61]]`.

13. MESSAGES > 100 CHARACTERS

PROBLEM STATEMENT: Perform element-wise comparison to identify messages longer than a threshold (e.g., >100 characters).



CO Untitled13.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾

☰

📄

⏮ ⏭

🔍

🔑

📁

📊

[1] ✓ 14s ▶ `from google.colab import files`
`uploaded = files.upload()`

⌵ ... Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[28] ✓ 0s ▶ `long_msgs = length_array > 100`
`print("Messages >100 chars:", np.sum(long_msgs))`

⌵ ... Messages >100 chars: 1744

14. CATEGORIES MESSAGE LENGTH

PROBLEM STATEMENT: Use NumPy where conditions to classify messages as “short”, “medium”, or “long”.



Untitled13.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

[1]
✓ 14s

```
from google.colab import files  
uploaded = files.upload()
```

 spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[29]
✓ 0s

```
df['category'] = np.where(df['length'] < 50, 'short',  
                          np.where(df['length'] < 100, 'medium', 'long'))  
print(df[['message', 'category']].head())
```



	message	category
0	Go until jurong point, crazy.. Available only ...	long
1	Ok lar... Joking wif u oni...	short
2	Free entry in 2 a wkly comp to win FA Cup fina...	long
3	U dun say so early hor... U c already then say...	short
4	Nah I don't think he goes to usf, he lives aro...	medium

15. NUMPY OPERATIONS

PROBLEM STATEMENT: use the NUMPY library to perform basic statistical calculations on dataset

CO

Untitled13.ipynb ☆

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text | ▶ Run all

☰

🔍

⏪ ⏩

🔗

📁

📊

[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[36]
✓ 0s

▶ import numpy as np

length_array = np.array(df['length'])

print("Mean:", np.mean(length_array))
print("Max:", np.max(length_array))
print("Min:", np.min(length_array))

... Mean: 78.97794544399304
Max: 910
Min: 2

16. WORD FREQUENCY ANALYSIS

PROBLEM STATEMENT: extract all words from messages labelled as “spam”.

MIT Academy of Engineering

 **Untitled13.ipynb**  

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

▼

Choose Files

 spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[46]
✓ 0s

 spam_words = ' '.join(df[df['label']=='spam']['clean_msg']).split()
word_series = pd.Series(spam_words)

print(word_series.value_counts().head(10))

▼

...	to	594
	a	331
	call	303
	you	259
	your	241
	free	188
	for	184
	the	181
	or	157
	now	156
	Name: count, dtype: int64	

17. UNIQUE VALUES

PROBLEM STATEMENT: Count how many messages are labeled as “spam” and “ham

 **Untitled13.ipynb** ☆

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[49]
✓ 0s

 print(df.nunique())

... label 2
message 5169
length 274
clean_msg 5141
category 3
label_num 2
dtype: int64

18. Find Average Word Count in Spam vs Ham Messages

Problem Statement : Calculate average number of words in spam and ham messages

 **Untitled13.ipynb** ☆
File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[50]
✓ 0s

df['word_count'] = df['message'].apply(lambda x: len(x.split()))

print(df.groupby('label')['word_count'].mean())

... label
ham 14.134632
spam 23.681470
Name: word_count, dtype: float64

19. Find Messages Containing Specific Keyword ("free")

PROBLEM STATEMENT: Count how many messages contain the word "free"

 **Untitled13.ipynb** ☆ ☁

File Edit View Insert Runtime Tools Help

🔍 Commands + Code ▾ + Text | ▶ Run all ▾



[1]
✓ 14s

from google.colab import files
uploaded = files.upload()

▼

Choose Files

spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[52]
✓ 0s

 free_msgs = df['clean_msg'].str.contains('free')
print("Messages containing 'free':", free_msgs.sum())

▼

... Messages containing 'free': 231

20. Find Top 5 Longest Spam Messages

PROBLEM STATEMENT: Display top 5 longest spam messages


Untitled13.ipynb
☆
☁

File Edit View Insert Runtime Tools Help

+ Code
+ Text
▶ Run all

☰

🔍

⏪ ⏩

🔍

🔑

📁

📊

[1]

✓ 14s

▶

```
from google.colab import files
uploaded = files.upload()
```

...

Choose Files

spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done

Saving spam (1).csv to spam (1).csv

[53]

✓ 0s

▶

```
top_spam = df[df['label']=='spam'].sort_values(by='length', ascending=False)
print(top_spam[['message', 'length']].head(5))
```

	message	length
1733	Hi, this is Mandy Sullivan calling from HOTMIX...	224
3718	Thanks for your ringtone order, reference numb...	197
2296	<Forwarded from 21870000>Hi - this is your Mai...	183
4904	Warner Village 83118 C Colin Farrell in SWAT t...	181
2246	Hi ya babe x u 4goten bout me?' scammers getti...	181

21. Compare Average Length Using NumPy (Spam vs Ham)

PROBLEM STATEMENT: Use NumPy to compare average lengths

CO

Untitled13.ipynb

☆ ↺ Saving...

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text | ▶ Run all

☰

🔍

⏏

🔗

🔑

📁

📊

[1] ✓ 14s

▶ `from google.colab import files`
`uploaded = files.upload()`

⌵

⋮ Choose Files spam (1).csv
spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[54] ✓ 0s

▶ `import numpy as np`

`spam_len = np.array(df[df['label']=='spam']['length'])`
`ham_len = np.array(df[df['label']=='ham']['length'])`

`print("Spam Avg Length:", np.mean(spam_len))`
`print("Ham Avg Length:", np.mean(ham_len))`

⌵

⋮ Spam Avg Length: 137.89127105666157
Ham Avg Length: 70.45925597874225

22. Create Binary Feature for Long Messages (>150 chars)

PROBLEM STATEMENT: Classify messages as long (1) or short (0)


Untitled13.ipynb
☆

File Edit View Insert Runtime Tools Help

+ Code
+ Text
▶ Run all

☰

🔍

<>

🔗

🔑

📁

📊

[1]

✓ 14s

▶

from google.colab import files
uploaded = files.upload()

Choose Files

spam (1).csv

spam (1).csv(text/csv) - 503663 bytes, last modified: 4/20/2026 - 100% done
Saving spam (1).csv to spam (1).csv

[55]

✓ 0s

▶

df['is_long'] = np.where(df['length'] > 150, 1, 0)

print(df[['message', 'is_long']].head())

...

	message	is_long
0	Go until jurong point, crazy.. Available only ...	0
1	Ok lar... Joking wif u oni...	0
2	Free entry in 2 a wkly comp to win FA Cup fina...	1
3	U dun say so early hor... U c already then say...	0
4	Nah I don't think he goes to usf, he lives aro...	0

Result / Observation :

- Dataset was successfully loaded and analyzed using NumPy and Pandas.
- Class distribution shows 86.6% ham and 13.4% spam — clear class imbalance.
- Spam messages are on average nearly 2x longer than ham messages (138 vs 71 chars).
- Spam messages contain more digits, uppercase letters, punctuation, and URLs.
- NumPy boolean masking and z-score methods efficiently identified outlier messages.
- Feature engineering columns (length, word count, digit count) can be used for ML.

Conclusion :

- NumPy and Pandas provide efficient and expressive methods for data analysis.
- The SMS Spam dataset clearly shows distinct statistical patterns between spam and ham.
- Feature engineering with Pandas str methods and NumPy math operations is powerful.

MIT Academy of Engineering

- Data cleaning (removing duplicates, null checks) is essential before analysis.
- Message length, digit count, and uppercase ratio are strong indicators of spam.